

Contents

Мануал к panel.py	1
1. Что это такое	1
2. Быстрый старт за 30 секунд	1
3. Установка	2
4. Запуск панели	2
5. Справочник команд	2
6. Подробности по командам	4
7. Состояние сессии	5
8. Артефакты	6
9. Формат YAML-конфигов	6
10. Интерпретация результатов	7
11. Частые сценарии	7
12. FAQ и устранение неполадок	8
13. Использование из Python (если панель не подходит)	8
14. Известные ограничения	9
15. Куда смотреть дальше	9

Мануал к panel.py

Единая интерактивная панель проекта 2039 «Механизмы цифрового забвения» МИЭМ НИУ ВШЭ · SISA Machine Unlearning · соответствие 152-ФЗ ст.14 / GDPR ст.17

1. Что это такое

panel.py — интерактивная консольная панель, которая объединяет в одном процессе полный цикл эксперимента с machine unlearning:

генерация → валидация → обучение → удаление → отчёт

В отличие от двух отдельных CLI (synthetic_data_generator/main.py и unlearning_system/cli.py), панель **сохраняет состояние между командами**: загруженный датасет, обученный ансамбль, текущий run_id и журнал сессии переживают между действиями. Это значит:

- удаление не перефитит модель заново,
- отчёт видит актуальный ансамбль,
- переключение на другой датасет автоматически сбрасывает кэш.

Панель запускается одним файлом, не требует веб-сервера и работает без GPU.

2. Быстрый старт за 30 секунд

```
cd /Users/hse hacker/Desktop/2039
```

```
python panel.py
```

В меню:

Ввести	Что произойдёт
1	Сгенерировать синтетический датасет (10 000 строк по умолчанию)
5	Обучить SISA-ансамбль из 5 шардов
8	Удалить по фильтру. Ввести: consent_given == False
10	Показать метрики текущего run
q	Выход

Всё. 4 команды — полный цикл. Артефакты — в unlearning_system/runs/<run_id>/.

3. Установка

Требования

- Python 3.12+
- RAM: 8 ГБ
- macOS 11+ / Ubuntu 20.04+ / Windows 10+
- GPU не нужен

Зависимости

```
cd /Users/hse hacker/Desktop/2039
```

```
# Генератор
```

```
cd synthetic_data_generator  
pip install -r requirements.txt
```

```
# Unlearning-система
```

```
cd ../unlearning_system  
pip install -r requirements.txt  
cd ..
```

Проверка

```
python panel.py --no-banner  
# должно показать меню; ввод 'q' завершает
```

4. Запуск панели

Базовые варианты

```
python panel.py # интерактивное меню + заставка  
python panel.py --no-banner # без ASCII-шапки
```

С предустановленным состоянием

```
# Сразу выбрать активный датасет
```

```
python panel.py --dataset synthetic_data_generator/data/my.csv
```

```
# Использовать свой YAML
```

```
python panel.py --unl-config configs/tuned.yaml  
python panel.py --gen-config configs/custom_gen.yaml
```

Режим отладки

```
PANEL_DEBUG=1 python panel.py  
# полный traceback при ошибках внутри действий
```

5. Справочник команд

В меню можно вводить номер пункта, русский, или английский алиас. Регистр не важен.

5.1. Работа с данными

№	Команда	Алиасы	Назначение
1	Сгенерировать	генерировать gen g	Новый синтетический CSV
2	Валидировать	валидировать val v	Проверить статистики существующего датасета
3	Инфо	инфо info i	Распечатать структуру и метрики датасета
4	Датасеты	датасеты ds list-data	Список data/, выбор активного

5.2. Модель

№	Команда	Алиасы	Назначение
5	Обучить	обучить train t	Обучить SISA-ансамбль на активном CSV
6	Оценить	оценить eval e	Переоценить текущий ансамбль на hold-out

5.3. Разобучение

№	Команда	Алиасы	Назначение
7	Удалить-ID	удалить-id uid unlearn-id	Удалить по списку user_id
8	Удалить-Ф	удалить-f uf unlearn-filter	Удалить по pandas.query()-фильтру

5.4. Отчётность и окружение

№	Команда	Алиасы	Назначение
9	Раны	раны runs	Все запуски в runs/
10	Отчёт	отчёт report r	Метрики выбранного run_id
11	Конфиг	конфиг config	Сменить YAML-конфиг
12	Показать-конфиг	show-config	Вывести активный конфиг
13	Журнал	журнал log j	Журнал операций сессии
0	Полный-цикл	полный-цикл all a	1→5→8→10 одной командой

5.5. Сервисные

Ввод	Назначение
h help помощь ?	Справка
c clear cls	Очистить экран
m menu меню	Перерисовать меню
q exit quit выход	Выход
(пустая строка)	Перерисовать меню
Ctrl+C	Прервать текущее действие, вернуться в меню
Ctrl+D	Завершить сессию

6. Подробности по командам

(1) Генерация датасета

Что спросит:

Количество записей [10000]: 5000

Seed генератора [42]: 42

Режим разметки: 1) rule 2) probabilistic [enter = из конфига]: 1

Что делает:

1. Прочитает `synthetic_data_generator/configs/default.yaml`
2. Сгенерирует датасет (ФИО, email, возраст, покупки, регион и т. д.)
3. Прогонит автоматическую валидацию (9 проверок)
4. Сохранит в `synthetic_data_generator/data/synthetic_dataset.csv`
5. Установит его активным, сбросит кэш пайплайна

Вывод:

✓ Сгенерировано: 5 000 строк × 21 признаков за 0.48 с

Пропущенные значения ✓ OK

Уникальность `user_id` ✓ OK

Распределения признаков ✓ OK

Ограничения (clip) ✓ OK

Корреляции ✓ OK

Форматы дат ✓ OK

Баланс классов ✓ OK

Уникальность email ✓ OK

Соответствие email←ФИО ✓ OK

Итог: ✓ валидация пройдена

✓ Сохранено: `.../synthetic_dataset.csv`

(5) Обучение SISA-ансамбля

Что делает:

1. Прочитает `unlearning_system/configs/default.yaml`
2. Загрузит активный CSV, стратифицированно разобьёт на train/test
3. Построит FeaturePipeline (z-score для числовых, one-hot для категориальных, bool→0/1)
4. Построит ShardPlan (FNV-1a хеш `user_id`, $S = 5$ шардов)
5. Независимо обучит 5 мини-моделей (PyTorch LogReg по умолчанию)
6. Посчитает baseline-метрики на hold-out
7. Создаст новый `runs/run_<timestamp>_<id>/` со всеми артефактами
8. Сохранит pipeline, baseline, `run_id` в состояние сессии

Вывод:

► Обучение SISA-ансамбля

Загрузка + предобработка...

✓ `train=4000 test=1000 features=21 shards=5`

Обучение ансамбля...

✓ Готово за 2.64 с, `run_id = run_20260417T105400_7cdc1b`

baseline accuracy : 0.9575

baseline F1 : 0.8874

baseline AUC : 0.9154

Размеры шардов : 812, 805, 791, 800, 792

(7) Удаление по `user_id`

Что спросит:

`user_id` (через пробел или запятую): `USR-000042, USR-000107 USR-000211`

Причина удаления [`user_request`]: 152-ФЗ ст.14

Что делает:

1. Построит `UnlearnRequest(user_ids=[...])`
2. Если ансамбль не обучен — автоматически вызовет пункт (5)
3. Вычислит `forget_mask` по train-набору
4. Определит множество затронутых шардов
5. Переобучит **только их**
6. Посчитает `before / after / Δ` метрик и `prediction_disagreement`
7. Допишет в тот же `run_id/metrics/` и `events.log`

(8) Удаление по фильтру

Синтаксис: `pandas.DataFrame.query()`. Примеры:

```
consent_given == False
age < 25 and region == 'Moscow'
purchase_count == 0 and days_since_last_purchase > 180
deletion_requested == True
is_premium == True and purchase_amount < 1000
region in ['Moscow', 'Saint-Petersburg'] and age >= 60
```

Важно: строковые значения — в одинарных кавычках, булевы — словами `True/False` без кавычек.

(10) Отчёт по run

Что спросит:

`run_id [run_20260417T105400_7cdc1b]: <enter>`

Что покажет:

- Все файлы из `runs/<id>/metrics/*.json` (`baseline`, `before_unlearn`, `after_unlearn`, `delta` — если были)
- Последние 5 строк `events.log`

(0) Полный цикл

Один пункт меню выполняет `1 → 5 → демо-удаление age < 25 and region == 'Moscow' → 10`. Удобно для:

- быстрого smoke-теста после обновления кода,
- демонстрации,
- сравнения конфигураций (запустить, сменить `unl_config`, запустить ещё раз).

7. Состояние сессии

В верхней части меню всегда видно:

Состояние сессии

```
Конфиг генератора :    synthetic_data_generator/configs/default.yaml
Конфиг unlearning :    unlearning_system/configs/default.yaml
Активный датасет   :    ../synthetic_dataset.csv
SISA-ансамбль      :    обучен
Текущий run_id     :    run_20260417T105400_7cdc1b
Baseline F1 / AUC  :    0.887 / 0.915
```

Как меняется:

Действие	Что сбрасывается
Генерация (1)	pipeline, baseline
Выбор активного датасета (4)	pipeline, baseline
Смена <code>unl_config</code> (11)	pipeline, baseline, <code>run_id</code> , <code>run_dir</code>
Ошибка train	pipeline не устанавливается, прежний сохраняется
Успешный train (5)	pipeline, baseline, <code>run_id</code> , <code>run_dir</code> обновляются
Unlearn (7/8)	Метрики переписываются в тот же <code>run_id</code>

8. Артефакты

Каждый train создаёт структуру:

```
unlearning_system/runs/run_20260417T105400_7cdc1b/
├─ manifest.yaml          ← конфиг, seeds, SHA-1 датасета, размеры шардов, backend
├─ pipeline.json          ← state_dict препроцессинга
├─ plan.npz              ← shard_of_row + n_shards
├─ learners/
│   ├── shard_00.joblib    ← state_dict каждого шарда
│   ├── shard_01.joblib
│   ├── shard_02.joblib
│   ├── shard_03.joblib
│   └── shard_04.joblib
├─ metrics/
│   ├── baseline.json
│   ├── before_unlearn.json ← появляются после первого unlearn
│   ├── after_unlearn.json
│   └── delta.json
└─ events.log             ← append-only аудит (JSONL)
```

events.log — это аудит-лог для **152-ФЗ**. Каждая запись содержит timestamp, тип события (train/unlearn), параметры запроса, причину, количество удалённых строк, затронутые шарды, wall time. Формат — одна JSON-строка на событие.

Дополнительно панель пишет **сессионный журнал** в `logs/panel_<timestamp>.log`.

9. Формат YAML-конфигов

unlearning_system/configs/default.yaml

```
experiment:
  run_root: runs           # корневая папка артефактов
  random_seed: 42
  test_size: 0.2           # hold-out

sharding:
  n_shards: 5              # число SISA-шардов
  shard_seed: 42           # seed FNV-1a маршрутизации

model:
  backend: logreg          # logreg | random_forest
  logreg:
    lr: 0.05
    weight_decay: 1.0e-4
    epochs: 200
    batch_size: 0          # 0 = full-batch
    device: cpu
    patience: 20
    tol: 1.0e-5
    class_weight: balanced
  random_forest:
    n_estimators: 100
    max_depth: 12
    min_samples_leaf: 2
    class_weight: balanced

preprocessing:
```

```
target_column: label
numeric_features: [age, purchase_amount, purchase_count,
                  avg_check, days_since_last_purchase]
boolean_features: [is_premium, is_subscribed, consent_given]
categorical_features: [region, device_type, preferred_category]
```

Чтобы переключиться на Random Forest — пункт меню 11, указать путь к копии YAML с backend: random_forest.

10. Интерпретация результатов

Панель подкрашивает Δ -метрики по порогам:

Зелёный (норма)	Жёлтый (внимание)	Красный (критично)
$\ \Delta\ < 0.01$	$0.01 \leq \ \Delta\ < 0.03$	$\ \Delta\ \geq 0.03$

Что делать по цвету

- **Всё зелёное** → удалённые данные не были критичны для предсказаний, разобучение корректно, retrain успешен.
- **Жёлтое по F1/recall** → удалена заметная часть положительного класса. Проверьте, не определяют ли удалённые строки сам класс (например, удаляете `consent == False`, а label основан на отсутствии согласия).
- **Красное** → значительная деградация. Варианты:
 - увеличить `weight_decay` и `epochs` (лучше регуляризация на меньшей выборке),
 - увеличить `n_shards` (меньше нужно переобучать, больше шума усредняется),
 - сделать периодический полный retrain после N unlearn-запросов,
 - проверить, что фильтр удаляет «правильные» строки (не весь класс).

Prediction disagreement (PD)

Доля тестовых объектов, где жёсткая метка изменилась после разобучения.

PD	Смысл
< 0.01	Модель действительно «не зависела» от удалённых
$0.01 \dots 0.03$	Заметное, но допустимое изменение
> 0.03	Удалённые данные были существенны, подумайте об архитектуре

11. Частые сценарии

Сценарий А. Базовый прогон

1 → 5 → 8 (фильтр) → 10

Сценарий Б. Сравнение LogReg vs Random Forest

1. 5 на `configs/default.yaml` (logreg) → запомнить `run_id_A`
2. 11 → выбрать `configs/rf.yaml`
3. 5 → получить `run_id_B`
4. 10 для обоих, сравнить `baseline.json`

Сценарий В. Массовое удаление

1. 1 с параметрами `count=50000`, `seed=42`
2. 5
3. 8 с фильтром `deletion_requested == True`
4. 10

Ожидаемо: удаление всех требующих удаления; AUC деградирует (в rule-based разметке именно эти строки — положительный класс).

Сценарий Г. «А что если удалить всё с `consent == False`?»

1. 1, 5
2. 8 → `consent_given == False`
3. 10: before/after/delta. В rule-based датасете F1 упадёт с ~0.89 до ~0.32 — потому что все положительные метки по построению содержат `consent_given == False` ИЛИ `deletion_requested == True`. После удаления «нет согласия» остаётся только подкласс «запрошено удаление», но и он часто коррелирует с `consent_given == False`.

Это диагностически полезно: показывает, что unlearning работает корректно.

Сценарий Д. «Забыть» конкретного клиента

1. 5
2. 7 → ввести `USR-000042`, причина 152-ФЗ ст.14
3. 10 → увидеть `deleted_rows: 1, affected_shards: [2]` (только один шард переобучен, стоимость $\approx 1/5$ от full retrain)

12. FAQ и устранение неполадок

Q: После 11 (смена конфига) пункт 10 говорит «нет файлов метрик». А: Смена конфига сбрасывает активный `run_id`. Запустите 5 (обучение) заново.

Q: Unlearn пишет «Ничего не указано для удаления». А: Вы не ввели ни `user_id` (в пункте 7), ни фильтр (в пункте 8). Пустая строка не считается валидным запросом.

Q: `age < 25 and region = 'Moscow'` — ошибка `SyntaxError`. А: Равенство в `pandas.query` — двойное: `==`. Одинарное `=` — присваивание, не поддерживается.

Q: Цвета не отображаются в терминале. А: Либо терминал не TTY (например, вывод перенаправлен в файл), либо используется старый Windows-терминал без поддержки ANSI. На Windows 10+ используйте Windows Terminal.

Q: Можно ли запускать панель в Jupyter? А: Нет. Панель использует `input()` и ожидает интерактивный TTY. В ноутбуке импортируйте `ExperimentPipeline` напрямую.

Q: Как добавить свой источник данных? А: Панель читает любой CSV/Parquet, лишь бы в нём были колонки из `preprocessing.*_features` и `preprocessing.target_column`. Поместите файл в `synthetic_data_generator/data/` и выберите через пункт 4.

Q: `Ctrl+C` что-то ломает? А: Нет. Прерывание возвращает в меню, состояние сессии сохраняется. Артефакты в `runs/` остаются.

Q: Как удалить накопившиеся старые runs? А: `rm -rf unlearning_system/runs/run_20260101*` — обычным `rm`. Панель не блокирует файлы.

Q: Меню отображается с разрывом в нумерации. А: Если в терминале менее 72 колонок, разделители могут переноситься. Расширьте окно до 80+ колонок.

13. Использование из Python (если панель не подходит)

```
from pathlib import Path
import sys
```



```

sys.path.insert(0, str(Path("unlearning_system").resolve()))

from ml.pipeline import ExperimentPipeline
from ml.unlearning import UnlearnRequest

pipe = (
    ExperimentPipeline
    .from_config("unlearning_system/configs/default.yaml")
    .load_dataset("synthetic_data_generator/data/synthetic_dataset.csv")
)
pipe.fit()

baseline = pipe.evaluate()
print(f"F1={baseline.f1:.3f}  AUC={baseline.auc:.3f}")

r = pipe.unlearn(UnlearnRequest(
    filter_query="consent_given == False",
    reason="GDPR art.17",
))
print(f"ΔF1={r.delta['f1']:+.4f}  deleted={r.deleted_rows}")

```

14. Известные ограничения

- **Нет slicing внутри шарда.** Удаление триггерит полный retrain шарда, а не откат до checkpoint'a. Это сознательное упрощение, см. THEORY.md.
 - **Нет MIA-верификации.** Количественной оценки «забыто ли» через атаку membership inference в прототипе нет.
 - **Одна модель на ансамбль.** Все шарды обучаются одним и тем же алгоритмом. Разнородные backbone'ы — extension.
 - **Старые runs без backend в manifest.yaml** отображаются как backend=? — это нормально, совместимость с предыдущими версиями формата.
-

15. Куда смотреть дальше

Документ	О чём
FULL_REPORT.pdf	Полный технический отчёт со всей теорией
REPORT.md	Краткий свод багов, найденных на аудите
unlearning_system/docs/THEORY.md	Детальная математика SISA
unlearning_system/README.md	README подсистемы unlearning
synthetic_data_generator/README.md	README генератора

Литература по Machine Unlearning: Bourtoule et al. 2021 (SISA); Guo et al. 2020 (Certified); Shokri et al. 2017 (MIA).

Документ обновлён: апрель 2026. Проект 2039 — МИЭМ НИУ ВШЭ, ТК 26.